Intelligent Contract Risk Analysis

Final Report: Milestone 1 & Milestone 2

Milestone 1: ML-Based Contract Risk Classification
Milestone 2: Agentic AI Legal Assistance System

Submitted in fulfillment of the B.Tech CSE (AI) Capstone Project

Newton School of Technology (NST), Sonipat

Team Members:
Ashutosh Singh (2401010109)
Ranvendra Pratap Singh (2401010373)
Shreya Suman (2401020068)
Avishkar Meher (2401010116)

April 2026

Declaration of Technical Integrity

The "No GenAI" Rule: We, the undersigned team members, formally affirm that the core logic of this project is our original work. All data parsing modules, Natural Language Processing pipelines, feature engineering implementations (TF-IDF vectorization), Scikit-Learn model training, and the Streamlit inference engine were manually and explicitly written and engineered by the team. The LangGraph agentic pipeline (Milestone 2) was designed and implemented independently, invoking external LLM APIs only for inference—not for generating any project code or logic. We have adhered strictly to the grading rubric throughout both milestones.

Contents

1 Problem Statement **4**

2 Data Description **4**

3 Exploratory Data Analysis **5**

4 Milestone 1: ML-Based Contract Risk Classification **6**

4.1 Preprocessing Pipeline 6

4.2 Feature Engineering: TF-IDF Vectorisation 6

4.3 Algorithms Evaluated 7

4.4 Optimisation Strategies 7

4.5 Evaluation Results 8

4.6 Milestone 1 Streamlit Dashboard 8

5 Milestone 2: Agentic AI Legal Assistance System **8**

5.1 Architecture Overview: From Static ML to Agentic AI 9

5.2 Node 1: Contract Domain Detection 9

5.3 Node 2: ML Risk Classification 9

5.4 Node 3: Domain-Aware RAG Research 10

5.5 Node 4: LLM Deep Analysis (Reason Node) 10

5.6 Cloud LLM — 5-Level Resilient Fallback Chain 11

5.7 Offline Mode — Local Ollama LLM 11

5.8 Report Generation 12

5.9 Milestone 2 Streamlit UI Enhancements 12

6 System Architecture & Code Structure **13**

7 Deployment **13**

8 Evaluation Against Grading Criteria **14**

9 Ethical Considerations & Responsible AI **14**

10 Team Contribution **15**

References **16**

1 Problem Statement

Modern corporate entities and legal firms process thousands of complex legal contracts quarterly, ranging from Non-Disclosure Agreements (NDAs) to multi-million dollar vendor agreements. The manual review of these documents is notoriously labour-intensive, time-consuming, and highly susceptible to human error. Legal professionals must meticulously scan dozens of pages of dense legal jargon to locate and evaluate specific clauses related to indemnification, financial liabilities, non-competes, and arbitrary termination conditions.

The objective of this project is to design, implement, and deploy an intelligent, end-to-end contract analysis system that operates across two progressive milestones:

- **Milestone 1:** A supervised machine-learning pipeline that accepts raw unstructured contract text, intelligently segments it into individual clauses, and accurately classifies each clause into one of three actionable risk categories (**High, Medium, Low**).
- **Milestone 2:** An agentic AI system built on **LangGraph** that extends the ML pipeline with domain detection, Retrieval-Augmented Generation (RAG) from a legal knowledge base, and structured LLM-generated analysis reports including plain-English summaries, negotiation strategies, and safer clause rewrites.

By flagging high-severity clauses and providing quantitative confidence scores and actionable legal guidance, this system drastically reduces the initial review burden for legal teams.

2 Data Description

Source & Provenance: The machine learning pipeline was trained and evaluated using the **Contract Understanding Atticus Dataset (CUAD) v1**, a high-quality open-source corpus curated by expert legal professionals for NLP research in the legal domain.

Corpus Structure: The raw dataset consists of a nested JSON architecture containing **510 unique, fully annotated commercial contracts**. Each document is structured hierarchically with 41 distinct legal clause types annotated via expert Question-Answer (QA) pairs.

Extraction Schema: Our custom data pipeline parsed the JSON natively. It filtered out all text blocks where the QA `is_impossible` flag was set to `true` (indicating that clause did not exist in the contract). For valid clauses, the system extracted the `clause_type` and the exact legal text string from the `answers` key. Final processed output: **7.1 MB CSV** containing 12,679 viable clause-level training examples.

Property	Value
Total Contracts	510 commercial contracts
Clause Types	41 distinct legal clause categories
Source Format	JSON (structured annotation)
Domain Coverage	Commercial, Employment, NDA, SaaS, Vendor, Lease
Annotation Method	Expert legal professionals
Total Extracted Clauses	12,679 (after filtering)
Processed CSV Size	7.1 MB

Table 1: CUAD v1 Dataset Summary

3 Exploratory Data Analysis

During the EDA phase, we systematically traversed the entire dataset to understand the fundamental statistical properties of the legal text corpus. Several critical insights shaped our methodology:

- **High Cardinality and Severe Class Imbalance:** The dataset natively contains 41 specific legal clause types. General boilerplate clauses (such as *Governing Law*, *Signatures*) appeared at drastically higher frequencies than severe penal clauses (like *Liquidated Damages* or *Uncapped Liability*).
- **Variable Context Lengths:** Significant variance was observed in text token lengths. Some clauses consisted of a single sentence, others (like IP Ownership clauses) spanned multiple dense paragraphs.
- **Contextual Ambiguity:** The same clause type could carry different effective risk levels depending on specific terms used (e.g., "mutual" vs. "unilateral" indemnity).

Strategic Risk Mapping

Predicting 41 heavily imbalanced classes using classical ML algorithms leads to severe overfitting. To resolve this, we engineered a mapping architecture that consolidated the 41 clause types into three actionable risk tranches:

1. **High Risk:** Clauses inflicting severe financial, operational, or legal threats (e.g., Uncapped Liabilities, Termination for Convenience, Non-Competes, IP Ownership Assignments, Indemnification).
2. **Medium Risk:** Clauses establishing operational friction or moderate constraints (e.g., Audit Rights, Insurance Requirements, Auto-Renewal, Arbitration clauses).

3. **Low Risk:** Standard administrative or boilerplate language carrying no active threat (e.g., Governing Law, Dates, Notice Requirements).

This consolidated the data into a healthy 3-class distribution:

Risk Class	Clause Count	Proportion
Low Risk	5,629	44.4%
Medium Risk	4,340	34.2%
High Risk	2,710	21.4%
Total	**12,679**	**100%**

Table 2: Risk Class Distribution After Consolidation

4 Milestone 1: ML-Based Contract Risk Classification

Objective: Design and implement a supervised ML-based contract analysis system that classifies clauses by risk level using classical NLP techniques, *without the use of Large Language Models or Generative AI.*

4.1 Preprocessing Pipeline

Raw legal text is inherently noisy and full of formatting artifacts. Our cleaning function executed the following systematic standardisations:

- **Case Normalisation:** All text transformed to lowercase for term uniformity.
- **Regex Scrubbing:** Stripped extraneous whitespace and special non-alphanumeric symbols, while intentionally retaining essential punctuation (`.,;:-'"()`) to preserve structural clause boundaries.
- **Clause Segmentation:** Contracts split on double-newline boundaries and sub-clause markers (numbered and lettered), retaining only segments exceeding 50 characters to filter out headings and formatting artifacts.

4.2 Feature Engineering: TF-IDF Vectorisation

To make text comprehensible to learning algorithms, we utilised **Term Frequency-Inverse Document Frequency (TF-IDF)**:

$$\text{TF-IDF}(t, d, D) = \underbrace{\frac{f_{t,d}}{\sum_{t' \in d} f_{t',d}}}_{\text{Term Frequency}} \times \underbrace{\log \frac{|D|}{|\{d \in D : t \in d\}|}}_{\text{Inverse Document Frequency}} \tag{1}$$

1. **Vocabulary Constraints:** `max_features=5000`. This dimensionality reduction prevents the model from memorising rare names or dates unique to specific contracts, forcing it to learn actual legal syntax.

2. **N-Gram Architecture:** `ngram_range=(1,2)`. Captures standalone legal terms (unigrams: "liability") and essential contextual pairings (bigrams: "breach of", "not exceed", "force majeure").

3. **Stop Word Removal:** Standard English stop words removed to eliminate structural noise.

4.3 Algorithms Evaluated

1. **Logistic Regression:** Selected for its efficiency in high-dimensional sparse spaces (5000-feature TF-IDF matrices). Provides calibrated probabilistic outputs via `predict_proba`, enabling the custom confidence thresholding feature in the UI.

2. **Decision Tree Classifier:** Evaluated to determine whether non-linear hierarchical branching rules outperformed linear boundaries.

3. **Random Forest Classifier:** Bootstrap-aggregation ensemble specifically evaluated to combat the high variance and overfitting observed in isolated Decision Trees.

4.4 Optimisation Strategies

1. **Pipeline Architecture (Preventing Data Leakage):** The `TfidfVectorizer` was encapsulated directly *inside* a Scikit-Learn `Pipeline` object alongside the classifier. This guarantees the vectorizer fits only on the training split of every cross-validation fold, never seeing holdout data.

2. **Class Weighting:** Logistic Regression was instantiated with `class_weight='balanced'`. This adjusts the internal cost function to penalise misclassifications on the minority High Risk class more severely than errors on the majority Low Risk class.

3. **5-Fold Stratified Cross-Validation:** Stratification ensured the relative class distribution of High/Medium/Low clauses remained strictly identical across all training and holdout folds, producing statistically reliable generalisation estimates.

Model Architecture	CV Accuracy	CV F1 (Weighted)	CV Precision
Logistic Regression	**88.56%**	**0.8859**	**0.8861**
Decision Tree Classifier	81.17%	0.8084	0.8091
Random Forest Classifier	81.38%	0.8058	0.8176

Table 3: Model Comparison — 5-Fold Stratified Cross-Validation

4.5 Evaluation Results

Why F1-Score? While Accuracy is reported, we relied on the **Weighted F1-Score** as the determinant metric. In legal risk analysis, False Negatives (missing a High-Risk liability cap) are legally disastrous. The F1 harmonic mean properly balances precision/recall tradeoffs.

Final Selection: Logistic Regression decisively outperformed tree-based methods with a near 89% F1-Score and was serialised as `best_model.pkl` for production inference.

4.6 Milestone 1 Streamlit Dashboard

The trained model was deployed as an interactive Streamlit dashboard featuring:

- Drag-and-drop PDF/TXT contract upload with instant text extraction via PyPDF2.
- Real-time metric cards showing High/Medium/Low clause counts and average confidence.
- Plotly Express visualisations: Risk distribution pie chart, confidence score histogram, risk intensity timeline, and top-keywords bar chart.
- Paginated, collapsible clause cards with risk labels, confidence percentages, and expandable full text.
- Confidence Threshold Slider: Dynamic filter to surface only high-confidence predictions (e.g., only flag clauses with >0.70 probability).
- One-click CSV export of the complete classified results.

5 Milestone 2: Agentic AI Legal Assistance System

Objective: Extend the ML classification system into a fully agentic AI legal assistance tool that autonomously reasons about contract risks, retrieves domain-specific legal best practices from a vector knowledge base, and generates structured, actionable reports using Large Language Models.

5.1 Architecture Overview: From Static ML to Agentic AI

Milestone 2 transforms the system from a **static ML classifier** into a **fully agentic AI pipeline**. The trained Logistic Regression model from Milestone 1 is now the *entry point* of a multi-step autonomous reasoning pipeline—not the final output.

The pipeline is orchestrated by a **LangGraph StateGraph** with four sequential nodes sharing a single immutable `AgentState` TypedDict object:

[S] →	**Node 1** Domain Detect	→	**Node 2** ML Classify	→
Node 3 RAG Research	→	**Node 4** LLM Reason	→	**[E]** Output

The key architectural property is `total=False` on the TypedDict, meaning each node writes only the keys it produces, enabling clean error propagation and full traceability at every stage. The compiled `StateGraph` is cached via `@lru_cache(maxsize=1)` to avoid re-compilation on every request.

5.2 Node 1: Contract Domain Detection

Before clause-level analysis begins, the system determines the contract's legal domain. This label is used to filter RAG results, inject domain-specific context into LLM prompts, and tag all outputs. Detection uses a two-stage algorithm:

1. **Stage 1 — Keyword Scoring (Zero API Cost):** A weighted regex signal table is applied against the first 3,000 characters of the document. Each of the 5 specific domains (NDA, Employment, Lease, SaaS, Vendor) has curated patterns with confidence weights on a 1–5 scale. Score ≥ 8 = high confidence; ≥ 4 = medium confidence.

2. **Stage 2 — LLM Fallback (When Score < 4):** A single lightweight LLM call is made using Groq (free tier) with `max_tokens=5` to disambiguate. The prompt instructs the model to return only one word from: `NDA | Employment | Lease | SaaS | Vendor | General`.

Six domains are supported: NDA, Employment, Lease, SaaS, Vendor, and General Commercial.

5.3 Node 2: ML Risk Classification

This node bridges Milestone 1 and Milestone 2. It loads the pre-trained Logistic Regression pipeline from `models/best_model.pkl` and:

- Segments the contract via `segment_clauses()`.
- Runs `model.predict()` and `model.predict_proba()` to get risk label and confidence.
- Applies the user-specified confidence threshold.
- Forwards **only High and Medium** risk clauses to downstream nodes, skipping Low-risk clauses to conserve LLM API tokens.
- Truncates clauses exceeding 2,000 characters to stay within LLM context limits.

5.4 Node 3: Domain-Aware RAG Research

For each flagged clause, the Research node retrieves the top-3 most relevant legal best-practice excerpts from the knowledge base. This retrieved context is injected into the LLM prompt to ground its reasoning in established legal standards and prevent hallucination.

The retrieval system uses a **3-Level Fallback Stack**:

1. **Level 1 — ChromaDB domain-filtered:** Highest quality. Queries the persistent vector store filtering by detected domain metadata. Embeddings produced by `all-MiniLM-L6-v2` (sentence-transformers; 384 dimensions). Distance metric: cosine similarity.
2. **Level 2 — ChromaDB full-collection:** If the domain has fewer results than the top-k target, the query is re-run across all domains.
3. **Level 3 — TF-IDF Fallback:** If ChromaDB is unavailable (e.g., first cold start), an in-memory TF-IDF retriever over `data/legal_best_practices.json` serves results using cosine similarity with a topic-hint injection mechanism.

The ChromaDB knowledge base contains **33+ domain-specific legal guidelines** covering all 6 domains: NDA (6 entries), Employment (6), Lease (6), SaaS (6), Vendor (4), and General Commercial (5).

5.5 Node 4: LLM Deep Analysis (Reason Node)

The Reason node dispatches each researched clause to a Large Language Model to produce an **8-field structured analysis**. Every LLM response is parsed into exactly the following fields:

Prompt Engineering: The system prompt injects: (1) a "Senior Legal Risk Analyst AI" persona, (2) the detected contract domain, (3) the ML risk label and confidence score, (4) the retrieved RAG excerpts, and (5) strict anti-hallucination rules forbidding citation

Field	Description
`plain_english_summary`	What the clause says at a 7th–8th grade reading level
`what_makes_it_risky`	Specific legal danger with a real-world consequence
`who_bears_the_risk`	Client / Vendor / Both / Third Party
`severity_rationale`	Why the ML model rated this clause High or Medium
`industry_standard_practice`	What a fair, balanced version looks like
`negotiation_tips`	2–3 specific, actionable negotiation points
`safer_rewrite`	Complete, ready-to-use formal replacement clause
`action_required`	Remove / Negotiate / Seek Legal Review / Accept with Caution

Table 4: LLM Analysis Output Schema — 8 Structured Fields

invention. Temperature is set to 0.15 for consistent, fact-grounded outputs. JSON output is enforced via `response_format={''type'': ''json_object''}`.

Fault-Tolerant Parser (`safe_parse_analysis()`): The JSON parser is designed to never raise an exception. It handles: `<think>...</think>` tags (Qwen/DeepSeek models), Markdown code fences (```` ```json ```), invalid JSON (regex substring search and retry), complete parse failures (returns defaults for all 8 fields), and backward compatibility with older 3-key response schemas.

5.6 Cloud LLM — 5-Level Resilient Fallback Chain

The cloud LLM client implements a 5-level fallback chain ensuring the system *always* returns a result:

#	Provider	API Key Required?	Model
1	**Pollinations AI**	No (free, no key)	`openai`
2	OpenRouter (Key 1)	Yes	`openai/gpt-oss-120b`
3	OpenRouter (Key 2)	Yes	`openai/gpt-oss-120b`
4	DeepSeek via OpenRouter	Yes	`deepseek/deepseek-chat`
5	Groq	Yes (free tier)	`llama-3.3-70b-versatile`
×	*All failed*	—	`safe_parse_analysis` returns defaults

Table 5: Cloud LLM Fallback Chain — Priority Order

Pollinations AI (Level 1) requires *no API key*, so the application works end-to-end out of the box, satisfying the free-tier only constraint.

5.7 Offline Mode — Local Ollama LLM

For complete privacy, the system supports a fully local offline mode powered by Ollama:

- If the Ollama daemon is not running, the system auto-starts it via `subprocess.Popen(['ollama 'serve'])`.
- Polls `GET /api/tags` every 500ms for up to 10 seconds to confirm readiness.
- Default model: `qwen3.5:2b` — compact, efficient, excellent JSON output.
- Timeout: 180 seconds to accommodate slow first-load of quantised models on CPU.

5.8 Report Generation

Every completed analysis produces two types of output:

1. **Structured JSON Report:** Machine-readable output containing the contract overview, detected domain, risk severity breakdown (High/Medium/Low counts), and per-clause analysis with all 8 LLM fields, ML metadata, and RAG references.
2. **PDF Report (fpdf2):** A professionally formatted PDF with a branded header, per-clause colour-coded risk badges (red/amber/green), all 8 analysis fields with section headers, and a legal disclaimer on every page.

5.9 Milestone 2 Streamlit UI Enhancements

The Milestone 2 UI adds a second tab ("AI Deep Analysis") to the existing Milestone 1 dashboard, featuring:

- Auto-detected domain badge with colour coding.
- Real-time clause-by-clause streaming progress indicator.
- Master-detail layout: clause list (left panel) and full analysis (right panel) with risk-severity coloured borders.
- JSON and PDF download buttons for the complete analysis report.
- Online/Offline toggle to switch between cloud LLM and local Ollama.
- RAG status pill showing ChromaDB (primary) or TF-IDF fallback (active retriever).

6 System Architecture & Code Structure

The repository is organised as a modular Python package:

```
Contract-Risk-Classification/
|-- app.py                    # Main Streamlit application
|-- rag_setup.py              # ChromaDB vector store builder
|-- requirements.txt          # Python dependencies
|-- run_app.sh                # Automated setup & launch script
|
|-- contract_agent/           # Core package (Milestone 2)
|   |-- workflow.py           # LangGraph StateGraph (4 nodes)
|   |-- core/
|   |   |-- state.py          # AgentState TypedDict
|   |   |-- domain.py         # Domain detection
|   |   |-- report.py         # JSON + Markdown reports
|   |   |-- pdf_report.py     # PDF export (fpdf2)
|   |-- llm/
|   |   |-- cloud.py          # 5-level LLM fallback chain
|   |   |-- local.py          # Ollama integration
|   |   |-- prompting.py      # Prompts + safe JSON parser
|   |-- retrieval/
|   |   |-- chroma.py         # ChromaDB retriever
|   |   |-- tfidf.py          # TF-IDF fallback retriever
|   |-- utils/
|       |-- ml.py             # Sklearn pipeline loader
|       |-- text.py           # clean_text, segment_clauses
|
|-- data/
|   |-- legal_best_practices.json # Static KB for TF-IDF
|   |-- processed/processed_contracts.csv # CUAD data (7.1 MB)
|
|-- models/
|   |-- best_model.pkl        # Trained LogReg pipeline (327 KB)
|
|-- src/                      # Jupyter notebooks (data pipeline)
    |-- 1_inspect.ipynb ... 6_phase2_agent_demo.ipynb
```

7 Deployment

The application is deployed on **Streamlit Community Cloud** at a publicly accessible URL, satisfying the mandatory hosting requirement (no localhost demos). Key deployment properties:

- Runs the complete ML Quick Screening pipeline without external dependencies.
- Automatically initialises the ChromaDB knowledge base on first run.
- Uses Pollinations AI free endpoint for LLM analysis with no API keys required.

Local Setup:

```
git clone https://github.com/ashyou09/Contract-Risk-Classification.git
cd Contract-Risk-Classification
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
python rag_setup.py       # Build ChromaDB (first run only)
streamlit run app.py
```

8 Evaluation Against Grading Criteria

Milestone 1 Criteria (25%)

Criterion	How Addressed
NLP & ML techniques	TF-IDF (bigrams, 5000 features) + Logistic Regression. 5-fold stratified CV. F1 = 0.8859.
Preprocessing & Feature Engineering	Custom clause segmentation, case normalisation, stop-word removal, bigram TF-IDF.
Evaluation metrics	Accuracy, Weighted F1, Precision, Recall, Confusion Matrix all computed.
UI Usability & Code structure	Streamlit dashboard with 6+ charts, confidence threshold slider, CSV export. Fully modular codebase.

Table 6: Milestone 1 Grading Criteria Mapping

Milestone 2 Criteria (30%)

9 Ethical Considerations & Responsible AI

This system was built with responsible AI principles embedded by design:

1. **Legal Disclaimer:** Every output (Markdown report, JSON, PDF, and UI sidebar) displays: *"This report is generated by an academic AI research prototype. It is NOT legal advice and may contain errors. Always consult a qualified legal professional before acting on any contract clause analysis."*

Criterion	How Addressed
Agentic Reasoning & Explainability	8-field structured LLM output with plain-English summaries, safer rewrites, and negotiation tips. Domain-aware prompting.
RAG Integration & State Management	ChromaDB with domain-filtered queries + TF-IDF fallback. Full `AgentState` TypedDict with error propagation across 4 nodes.
Ethical Awareness & Responsible AI	Legal disclaimer on all outputs. Anti-hallucination prompt guardrails. ML confidence displayed. Offline mode for privacy.
Deployment	Live public URL on Streamlit Community Cloud. Works with zero API keys via Pollinations AI.

Table 7: Milestone 2 Grading Criteria Mapping

2. **Confidence Transparency:** ML confidence scores are displayed alongside every risk classification, enabling users to assess model certainty before acting.
3. **Anti-Hallucination Prompting:** The LLM is explicitly instructed not to invent legal citations and to reference only provided RAG knowledge base excerpts.
4. **Human Oversight:** The system presents analysis and evidence. All final legal decisions explicitly remain with the human user.
5. **Privacy Preservation:** Offline mode (Ollama) enables complete on-device processing with no data leaving the user's machine.
6. **No Over-Reliance Guardrail:** The LLM prompt explicitly forbids responses consisting solely of "consult a lawyer" without substantive analysis.

10 Team Contribution

The workload was systematically divided across both milestones:

Ashutosh Singh (2401010109) Developed the custom data preprocessing scripts responsible for parsing the CUAD JSON architecture. Implemented the core risk-label mapping logic (41 classes to 3-tier system). Architected the base Scikit-Learn Pipeline preventing data leakage, RAG and Local Ollama integration, trained and evaluated all ML models, and contributed to the overarching Streamlit UI dashboard structure.

Ranvendra Pratap Singh (2401010373) Implemented the TF-IDF feature engineering strategy. Co-defined the pipeline architecture. Designed and implemented the Lang-

Graph StateGraph (Milestone 2), the ChromaDB RAG integration, the 5-level cloud LLM fallback chain, Open Source LLM integration, and the primary architect of the Streamlit UI for both milestones.

Shreya Suman (2401020068) Led execution of the Scikit-Learn cross-validation pipeline. Performed Stratified K-Fold evaluation to establish champion model metrics. Led the report generation module (JSON, Markdown, PDF outputs) and contributed to ChromaDB Logic and involved heavily to Streamlit UI and front-end design and integration.

Avishkar Meher (2401010116) Led quality assurance checks and end-to-end integration testing across both milestones. Performed deployment validation on Streamlit Community Cloud. Participated in code review, workflow refinement, and project documentation.

References

[1] Hendrycks, D., Burns, C., Chen, A., & Ball, S. (2021). *CUAD: An Expert-Annotated NLP Dataset for Legal Contract Review*. arXiv:2103.06268.

[2] Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.

[3] LangChain, Inc. (2024). LangGraph: Building Stateful Multi-Actor Applications with LLMs. https://github.com/langchain-ai/langgraph

[4] Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. *Proceedings of EMNLP 2019*.

[5] Chroma. (2024). ChromaDB: The AI-native open-source embedding database. https://www.trychroma.com/

[6] Lewis, P., et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *Advances in NeurIPS 2020*.

[7] Sparck Jones, K. (1972). A statistical interpretation of term specificity and its application in retrieval. *Journal of Documentation*, 28(1), 11–21.

[8] Streamlit Inc. (2024). Streamlit — The fastest way to build data apps. https://streamlit.io/